Diseño de APIs y REST

Principios API First, recursos y URIs

IBERO Puebla — Otoño 2026
Mtro. Rafael Pérez Aguirre

DOS FORMAS DE EMPEZAR

Code-first vs. API First

CODE-FIRST

Se empieza a programar el backend y la forma de la API va emergiendo sobre la marcha, como efecto secundario del código.

API FIRST

Se diseña el contrato de la API (qué recursos existen, qué operaciones soportan) antes de escribir una sola línea de implementación.

¿Por qué conviene?

Trabajo en paralelo

El equipo de frontend puede empezar contra un contrato acordado, sin esperar a que el backend exista.

Errores más baratos

Los problemas de diseño se detectan en una conversación, no en un refactor de código ya escrito.

Contrato documentable

El diseño se puede formalizar y documentar; lo veremos con Swagger/ReDoc.

Recursos: sustantivos, no verbos

MAL DISEÑO — VERBOS EN LA URI

```
GET  /getSongs
POST /createSong
POST /deleteSong?id=5
```

BUEN DISEÑO — RECURSO + MÉTODO

```
GET    /songs
POST   /songs
DELETE /songs/5
```

La URI identifica la cosa (el recurso). El método HTTP indica la acción.

DEL CONJUNTO A LA PARTE

Colección, elemento y anidamiento

`/songs`	Colección completa (plural)
`/songs/5`	Un elemento específico dentro de la colección
`/artists/12/songs`	Anidado: canciones que pertenecen al artista 12

Anidar solo cuando el recurso hijo no existe fuera del padre. 3+ niveles es señal de mal diseño.

REGLAS DEL CAMINO

Convenciones de URIs

CONVENCIÓN	CORRECTO	INCORRECTO
Sustantivos en plural	`/songs`	`/song`
Minúsculas	`/songs`	`/Songs`
Kebab-case	`/top-tracks`	`/topTracks`
Sin verbos	`/songs/5`	`/getSong/5`
Sin slash final	`/songs`	`/songs/`
Versionado con prefijo	`/api/v1/songs`	`/songs?version=1`

DOS FORMAS DE CALIFICAR UNA URI

Path params vs. query params

Path param

Identifica CUÁL recurso.
No es opcional: /songs/5 apunta a un elemento específico.

Query params

Modifican CÓMO se devuelve la colección: filtrado, orden, paginación. Son opcionales.

```
GET /songs?genre=pop&min_popularity=70&sort=popularity&page=2
```

DE LA OPERACIÓN AL ENDPOINT

CRUD → método + URI

OPERACIÓN	MÉTODO	URI	CÓDIGO
Listar canciones	GET	/songs	200
Obtener una canción	GET	/songs/{id}	200
Crear una canción	POST	/songs	201
Reemplazar completa	PUT	/songs/{id}	200
Actualizar un campo	PATCH	/songs/{id}	200
Borrar una canción	DELETE	/songs/{id}	204

ACTIVIDAD

El dominio: catálogo de Spotify

YA LO CONOCEN

Es el mismo catálogo de 30,000 canciones que ya trabajaron con funciones en Python puro: buscar por artista, filtrar por género, calcular estadísticas.

HOY

Esas mismas funciones se convierten en el mapa de una API: cada pregunta que ya resolvían en código es hoy un endpoint por diseñar.

ACTIVIDAD

¿Qué deben producir?

1. Lista de recursos identificados (mínimo 3, ej. songs y artists).
2. Endpoints CRUD para cada recurso (solo los que tengan sentido).
3. Al menos un endpoint anidado que exprese una relación real del dominio.
4. Un endpoint de colección con query params de filtrado/orden/paginación.
5. Para cada endpoint: método, URI y código de éxito.